

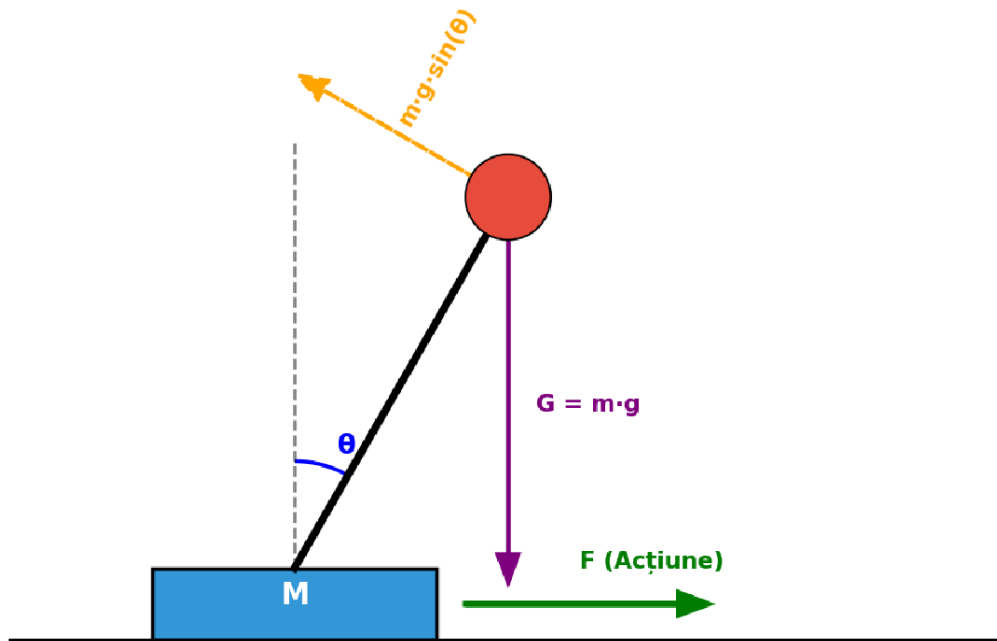
Controlul Automat al unui Pendul Invers utilizând Algoritmul Q-Learning

Proiect realizat de :

Popovici Andrei
Duda Andrei-Ionuț
Jireadă Teodor

1. Descrierea problemei considerate

Problema "Pendulului Invers" (cunoscută în literatura de specialitate sub numele de *CartPole*) este o problemă clasică în domeniul teoriei controlului și al roboticii. Sistemul este compus dintr-un stâlp (pendul) atașat printr-o articulație neacționată de un cărucior (platformă), care se deplasează de-a lungul unei șine orizontale fără frecare.



Obiectivul: Scopul este de a menține pendulul într-o poziție verticală (echilibru instabil) cât mai mult timp posibil. Deoarece gravitația tinde să doboare pendulul, sistemul trebuie să aplice forțe asupra căruciorului (stânga sau dreapta) pentru a contracara căderea.

Provocarea: Problema este dificilă deoarece pendulul este instabil prin natura sa. O mică deviație de la verticală se amplifică rapid sub influența gravitației. Agentul de control trebuie să decidă în timp real direcția de mișcare bazându-se doar pe poziția curentă și viteza sistemului.

2. Aspecte teoretice privind algoritmul

Pentru rezolvarea acestei probleme s-a utilizat **Învățarea prin Consolidare (Reinforcement Learning)**, specific algoritmul **Q-Learning**.

Conceptul de bază

Învățarea prin consolidare presupune existența unui **Agent** care interacționează cu un **Mediu**. Agentul ia o acțiune, iar mediul răspunde cu o nouă stare și o recompensă (pozitivă sau negativă).

Algoritmul Q-Learning

Q-Learning este o metodă "model-free" (nu necesită un model fizic predefinit al mediului) care învață o funcție $Q(s,a)$. Această funcție estimează "calitatea" (valoarea) efectuării unei acțiuni a într-o stare s .

Ecuția Bellman pentru actualizare: Agentul își actualizează cunoștințele folosind formula:

$$Q^{new}(S_t, A_t) \leftarrow (1 - \underbrace{\alpha}_{\text{learning rate}}) \cdot \underbrace{Q(S_t, A_t)}_{\text{current value}} + \underbrace{\alpha}_{\text{learning rate}} \cdot \left(\underbrace{R_{t+1}}_{\text{reward}} + \underbrace{\gamma}_{\text{discount factor}} \cdot \underbrace{\max_a Q(S_{t+1}, a)}_{\text{estimate of optimal future value}} \right)$$

new value (temporal difference target)

Unde:

- $Q(s,a)$: Valoarea curentă a perechii stare-acțiune.
- α (Alpha): **Rata de învățare** (cât de repede acceptăm informații noi).
- R : **Recompensa** primită imediat.
- γ (Gamma): **Factorul de discount** (importanța recompenselor viitoare).
- $\max Q(s',a')$: Cea mai bună recompensă estimată pentru starea următoare.

Strategia Epsilon-Greedy: Pentru a balansa explorarea (încercarea de mișcări noi) cu exploatarea (folosirea a ceea ce știm deja), folosim parametrul ϵ (Epsilon). La început, ϵ este mare (mișcări aleatoare), iar pe parcurs scade, agentul bazându-se pe Q-Table.

3. Modalitatea de rezolvare

Implementarea a fost realizată în limbajul **Python**, utilizând următoarele biblioteci:

- **Pygame:** Pentru simularea grafică și vizualizarea mișcării în timp real.
- **NumPy:** Pentru manipularea matricelor (Q-Table) și calcule matematice.
- **Math:** Pentru funcții trigonometrice necesare fizicii.

Discretizarea Stărilor

Sistemul fizic este continuu (poziția poate fi 1.2345 metri), dar Q-Table-ul este o matrice finită. Pentru a aplica Q-Learning, am "discretizat" spațiul stărilor. Am împărțit valorile continue în intervale (bins):

1. Poziția căruciorului (Cart Position)
2. Viteza căruciorului (Cart Velocity)
3. Unghiul pendulului (Pole Angle)
4. Viteza unghiulară (Pole Angular Velocity)

Astfel, o infinitate de stări posibile este redusă la un număr finit de combinații de indici în tabelul Q.

4. Listarea părților semnificative din codul sursă însoțite de explicații și comentarii

Configurarea Spațiului de Stări (Discretizarea)

Pentru a utiliza un tabel Q, trebuie să transformăm valorile reale (continue) în indici (discreți). Am ales următoarele granițe:

- Poziția (x): Intervalul $[-3.0, 3.0]$ împărțit în 10 secțiuni.
- Unghiul (theta): Intervalul $[-1.5, 1.5]$ radiani împărțit în 20 de secțiuni.

Python

```
bins_x = np.linspace(-3.0, 3.0, 10)
bins_theta = np.linspace(-1.5, 1.5, 20)

nr_stari_x = len(bins_x) + 1          # 11 stări
nr_stari_theta = len(bins_theta) + 1 # 21 stări
nr_actiuni = 2                       # LEFT, RIGHT
```

Acest lucru generează un Q-Table cu dimensiunile (11, 21, 2) umplut inițial cu zero-uri.

Definirea Mediului (HammerEnvironment)

În acest bloc de cod setăm constantele fizice: gravitația, masele componentelor și lungimea pendulului.

Funcția de Reset

Este esențială pentru start. Agentul primește valori aleatorii mici pentru a învăța din diverse unghiuri.

- Componente Stare: Poziție obiect (x), viteză cărucior (x_punct), unghi bară (theta), viteză unghiulară (theta_punct).

Cod Sursă - Reset:

Python

```
self.state = np.random.uniform(low=-0.05, high=0.05,  
size=(4,))
```

Funcția Step și Dinamica Sistemului

Această funcție calculează mișcarea fizică la fiecare pas.

1. Verificare Acțiune:
 - Dacă acțiunea este RIGHT -> forța este pozitivă.
 - Dacă acțiunea este LEFT -> forța este negativă.
2. Calcul Forțe:
 - temp: Forța aplicată adunată cu forța centrifugă.
 - thetaacc: Accelerația unghiulară (cum cade/se ridică ciocanul).
 - xacc: Accelerația liniară a căruciorului.
3. Integrarea Euler: Se folosește un pas de timp (tau) de 0.02 secunde (50 Hz) pentru a actualiza poziția.

Algoritmul Q-Learning și Formula Bellman

Învățarea se bazează pe ajustarea valorilor din tabel folosind experiența dobândită.

Formula matematică utilizată în cod: $Q_{\text{nou}}(st, At) = (1 - \text{Alpha}) * Q(st, At) + \text{Alpha} * (\text{reward} + \text{Gamma} * \max_a Q(st+1, a))$

Parametrii de antrenament:

- ALPHA (Learning Rate): 0.1
- GAMMA (Factor Discount): 0.98
- EPSILON (Probabilitatea de explorare): 1.0 (scade treptat).

Buclo de Antrenament (Training Loop)

Antrenamentul se desfășoară pe parcursul a 1000 de episoade.

Logica implementată:

1. **Seleție Acțiune:** Dacă un număr random este mai mic decât EPSILON, se alege o mișcare haotică (explorare). Altfel, se alege cea mai bună acțiune din tabel (expert/argmax).
2. **Actualizare Tabel:**
 - Se citește old_value (valoarea veche din tabel).
 - Se calculează next_max (cea mai bună perspectivă viitoare).
 - Se aplică formula Bellman pentru a obține new_value.
 - Se salvează noua valoare în tabel.
3. **Epsilon Decay:** După fiecare episod, valoarea EPSILON scade (se înmulțește cu 0.995) pentru a forța agentul să folosească experiența acumulată în loc de mișcări random.

Python

```
# Training
```

```
ALPHA = 0.1
```

```
GAMMA = 0.98
```

```
EPSILON = 1.0
```

```
EPSILON_DECAY = 0.995
```

```
env = HammerEnvironment()
```

```
print("Antrenamentul cu 1000 de episoade:")
```

```

for episod in range(1000):

    state_real = env.reset()
    # pornim de la valori random nu din pozitie 0,0,0,0 pt
    ca agentul sa poata invata

    state_disc = get_state(state_real)
    done = False
    total_reward = 0

    while not done:
        if np.random.random() < EPSILON:
            action = np.random.randint(0, 2) # alegem stg,
            dreapta random
        else:
            action = np.argmax(q_table[state_disc]) #
            expert -> alegem ce e mai bun

        # Aplicam actiunea
        next_state_real, reward, done = env.step(action)

        # Discretizam pt Q-Table
        next_state_disc = get_state(next_state_real)

        old_value = q_table[state_disc + (action,)] # Q(st,
        at)
        next_max = np.max(q_table[next_state_disc]) #
        max(Q(st+1, a))

        new_value = (1 - ALPHA) * old_value + ALPHA *
        (reward + GAMMA * next_max)

```

```
# Punem in Q-Table
q_table[state_disc + (action,)] = new_value

state_disc = next_state_disc
total_reward += reward

if EPSILON > 0.01:
    # Scadem Epsilon pt a "forta" sa foloseasca Q-Table
    # dupa ceva timp pt actiuni, sa nu mai fie random
    EPSILON *= EPSILON_DECAY

if (episod + 1) % 100 == 0:
    print(f"Episod: {episod + 1}, Scor Total:
    {total_reward:.2f}, Epsilon: {EPSILON:.2f}")
```


1. Selectăm granița $[-3, 3]$ axa Ox
granița $[-1.5, 1.5]$

$\Rightarrow$ $\text{bins-x} = \text{np.linspace}(-3.0, 3.0, 10)$
 $\text{bins-theta} = \text{np.linspace}(-1.5, 1.5, 20)$

2. Obținem nr. stări în care ne aflăm:

$\text{nr-stari-x} = \text{len}(\text{bins-x}) + 1$ #11 #21
 $\text{nr-stari-theta} = \text{len}(\text{bins-theta}) + 1$
 $\text{nr-actiuni} = [\text{LEFT}, \text{RIGHT}]$ #2
 $\Rightarrow Q\text{-Table}(11, 21, 2)$ umplut cu zero-uri.

3. Setăm $\text{HammerEnvironment}(\text{+threshold-radius}, \text{threshold-x})$
° gravitație, unghi, lungime, etc.

4. Funcția de reset (condițional de start)

> Agentul trebuie să primească mișcările potrivite din $Q\text{-Table}$, astfel ca o nouă stare sau genera valori random. uniform pt obiect.

$\text{self.state} = \text{np.random.uniform}(\text{low}=-0.05, \text{high}=0.05, \text{size}=(4,))$

poziție obiect (x),
viteză corucior ($\dot{x}$)
unghi bară (θ)
viteza unghiulară ($\dot{\theta}$)

5. Funcția de step

$\text{step}(\text{self}, \text{action})$

1. Verificăm dacă $\text{action} == \text{RIGHT} \Rightarrow \text{force} = \text{self.force} + \text{mag}$
 $\text{action} == \text{LEFT} \Rightarrow \text{force} = -\text{self.force} - \text{mag}$

2. Calculăm forțele necesare: temp (forța aplicată + forța centrifugă),
 theta_acc (cădere / ridicare ciocanului)
 x_acc : (accelelație corucior)

și actualizăm pozițiile. (Integrare Euler)

$\text{self.tau} = 0.02 \Rightarrow 50 \text{ Hz}$

6. Algoritmă Q-Learning

Formula Bellman

$$Q^{new}(s_t, a_t) = Q(s_t, a_t) + \alpha (r_{t+1} + \gamma \cdot \max_a Q(s_{t+1}, a) - Q(s_t, a_t))$$

α = learning rate (ALPHA)

r_t = reward

γ = factor discount (GAMMA)

ϵ = probabilitatea de explore

TRAINING:

ALPHA = 0.1

GAMMA = 0.98

EPSILON = 1.0

EPSILON-DECAY = 0.995

env = HammerEnvironment()

for (episod) in range(1000):

state-real = env.reset()

state-disc = get-state(state-real)

done = False

while not done:

if np.random.random() < EPSILON:

action = np.random.randint(0, 2) # alegem stg, dreapta random

else:

action = np.argmax(Q-table[state-disc]) # expert → alegem ce e mai bun

Aplicați acțiunea

next-state-real, reward, done = env.step(action)

Discretizăm pt Q-Table

next-state-disc = get-state(next-state-real)

old-value = Q-table[state-disc + (action,)] # $Q(s_t, a_t)$

next-max = np.max(Q-table[next-state-disc]) # $\max_a Q(s_{t+1}, a)$

new-value = (1 - ALPHA) * old-value + ALPHA * (reward + GAMMA * next-max) # max

Punem în Q-Table

Q-table[state-disc + (action,)] = new-value

state-disc = next-state-disc

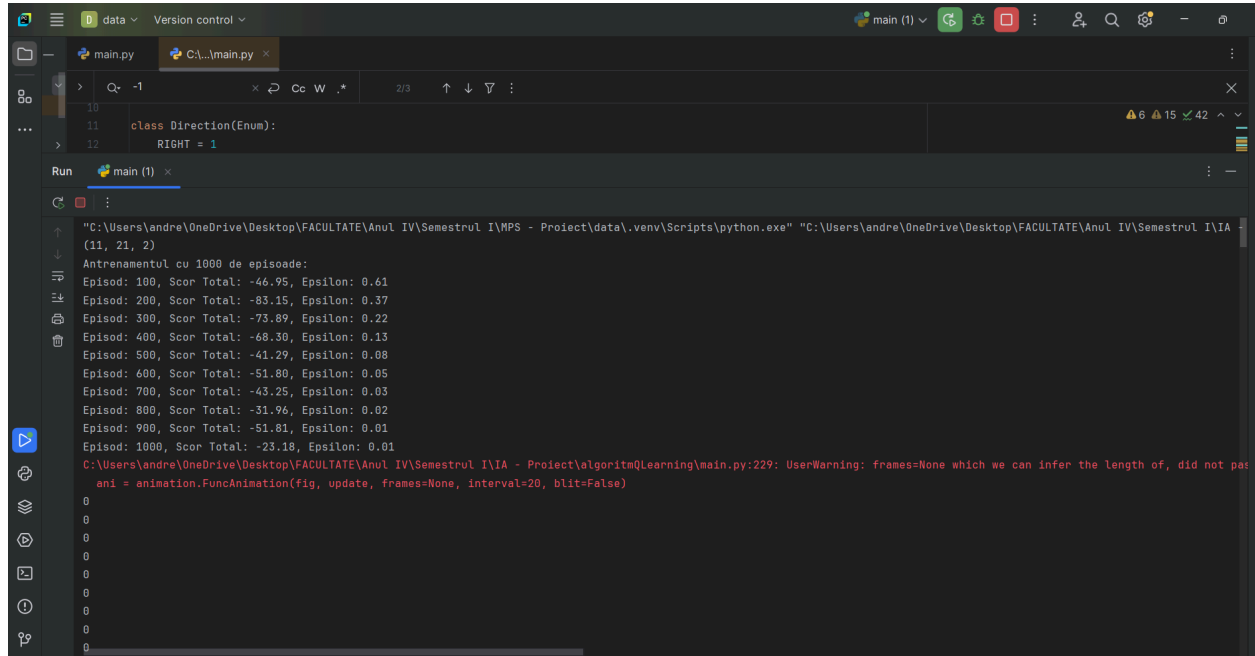
total-reward += reward

if EPSILON > 0.01:

EPSILON *= EPSILON-DECAY # Scădem EPSILON pt a ajunge "să folosească Q-Table după ce a învățat acțiunea, să nu mai fie random

5. Rezultatele obținute prin rularea programului în diverse situații, capturi ecran și comentarii asupra rezultatelor obținute

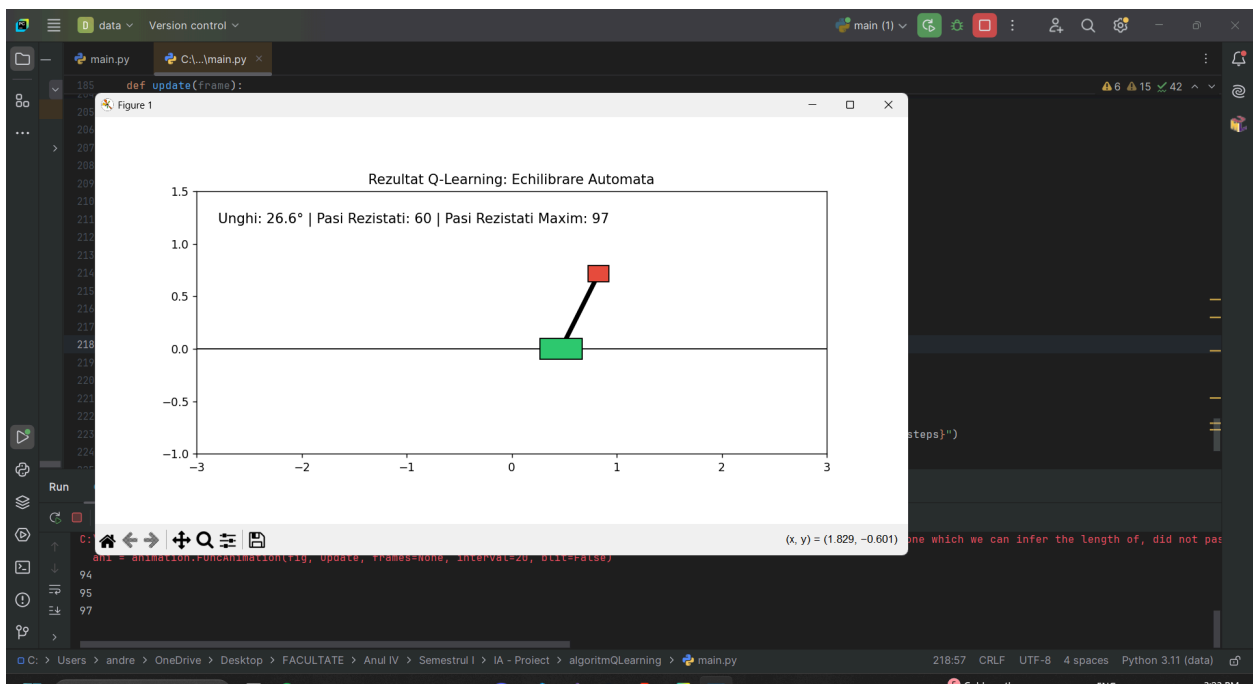
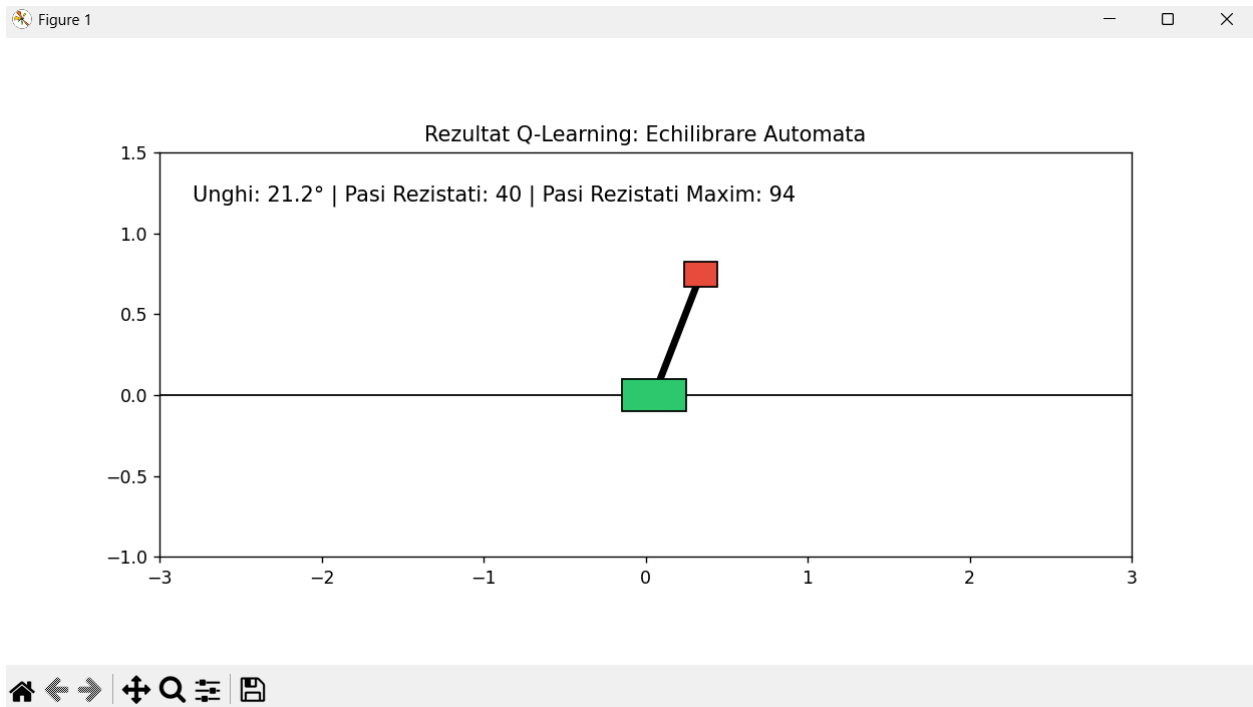
Antrenamentul cu cele 1000 de episoade ne arată că total_reward devine din ce în ce mai mare.



The screenshot shows a Python IDE with a file explorer on the left, a code editor in the center, and a run console at the bottom. The code editor displays a Python script with a class definition for 'Direction' and a variable 'RIGHT'. The run console shows the output of the program, which includes a list of training results for 1000 episodes. The results show a steady increase in total reward and a decrease in epsilon over time, indicating successful training.

```
class Direction(Enum):  
    RIGHT = 1  
  
Run  
"C:\Users\andre\OneDrive\Desktop\FACULTATE\Anul IV\Semestrul I\MP5 - Proiect\data\.venv\Scripts\python.exe" "C:\Users\andre\OneDrive\Desktop\FACULTATE\Anul IV\Semestrul I\IA - (11, 21, 2)  
Antrenamentul cu 1000 de episoade:  
Episod: 100, Scor Total: -46.95, Epsilon: 0.61  
Episod: 200, Scor Total: -83.15, Epsilon: 0.37  
Episod: 300, Scor Total: -73.89, Epsilon: 0.22  
Episod: 400, Scor Total: -60.30, Epsilon: 0.13  
Episod: 500, Scor Total: -41.29, Epsilon: 0.08  
Episod: 600, Scor Total: -51.80, Epsilon: 0.05  
Episod: 700, Scor Total: -43.25, Epsilon: 0.03  
Episod: 800, Scor Total: -31.96, Epsilon: 0.02  
Episod: 900, Scor Total: -51.81, Epsilon: 0.01  
Episod: 1000, Scor Total: -23.18, Epsilon: 0.01  
C:\Users\andre\OneDrive\Desktop\FACULTATE\Anul IV\Semestrul I\IA - Proiect\algorithm\learning\main.py:229: UserWarning: frames=None which we can infer the length of, did not pos  
ani = animation.FuncAnimation(fig, update, frames=None, interval=20, blit=False)  
0  
0  
0  
0  
0  
0  
0  
0  
0  
0
```

Versiunea v1

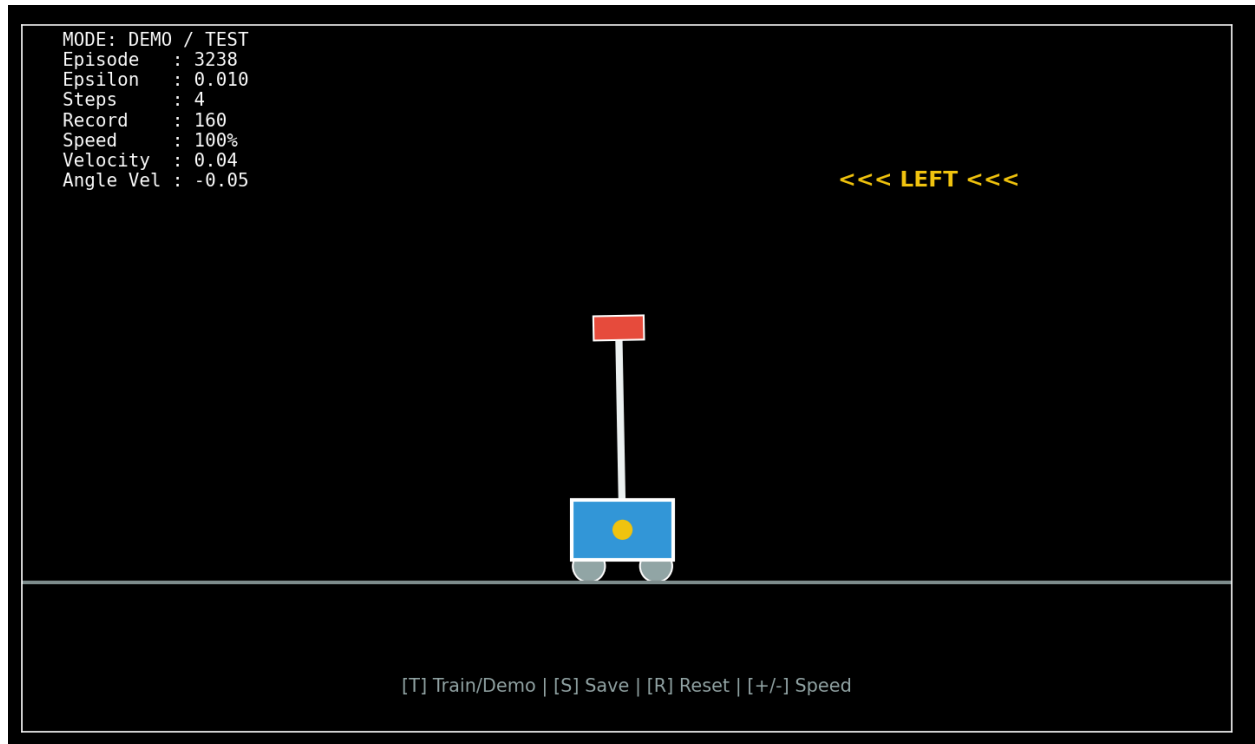
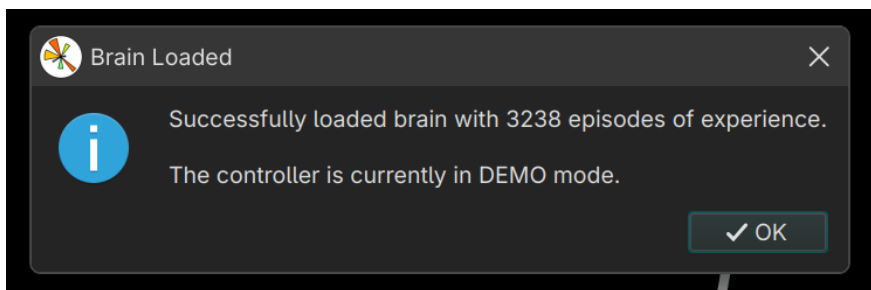


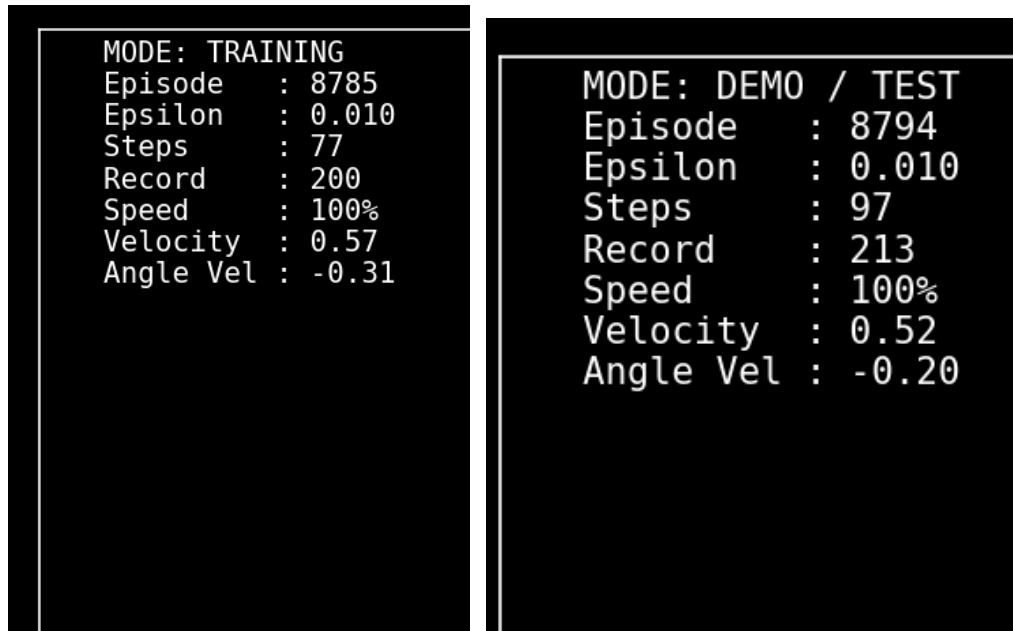
După cele 1000 de episoade, animația pornește și contorizează recordul de pași rezistați de obiect după ce a devenit (EXPERT). Din cele 2 screenshot-uri se poate observa și în consolă evoluția numărului de pași maximi. Această valoare va suferi o plafonare după un număr mare de episoade (1500+).

Versiunea v2:

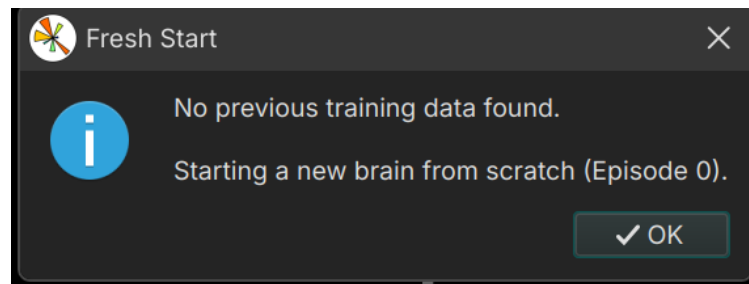
Într-o versiune ulterioară a aplicației am adăugat funcționalități suplimentare pentru UI și pentru memorarea episoadelor. Aplicația prezintă acum informații suplimentare despre modul de antrenare și interacțiune cu utilizatorul precum:

- Mod de antrenare sau de testare [Tasta 'T']
 - Comparativ cu v1, acum modul EXPERT îl poate defini utilizatorul în funcție de câte episoade consideră necesare pentru rulare.
 - Antrenarea se poate observa vizual cu fiecare episod.
 - În timpul modului de TEST antrenarea se oprește; numărul de episoade rămâne același

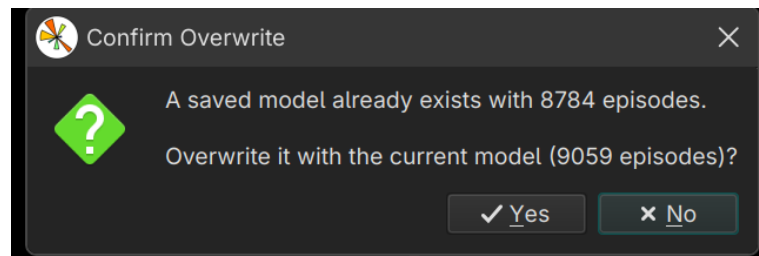




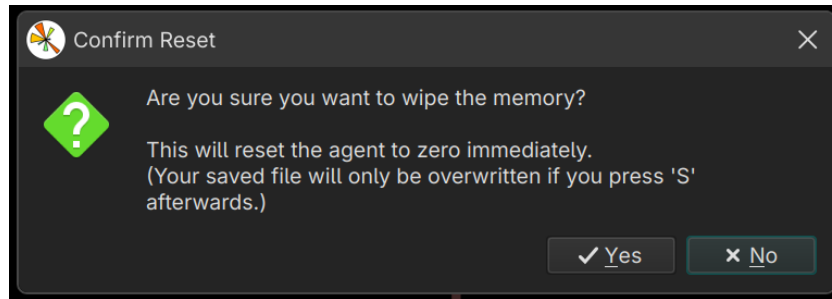
- Salvarea datelor de antrenament [Tasta 'S']
 - Dacă nu există date de antrenament (fișierul q_table.npy) programul pornește cu un set nou de date



- Salvarea datelor de petrece **doar** la apăsarea tastei 'S'. Aceasta va scrie peste vechiul set de date



- Resetarea datelor [Tasta 'R']
 - Dacă se dorește reluarea antrenamentului curent, datele se pot reseta. Salvarea nu se petrece decât la cererea explicită cu tasta 'S'



- Viteza animației de antrenare [Tastele '+' / '-']
 - Se poate modifica viteza cu care se vizualizează antrenarea și a animațiilor.
 - Cel mai rapid mod de a rula un algoritm deja antrenat rămâne cel din v1, în care nu se vizualizează efectiv fiecare episod al antrenamentului.



6. Concluzii

Algoritmul Q-learning permite agentului să dezvolte o politică optimă pentru menținerea echilibrului unui pendul invers, transformând un sistem instabil într-unul controlat.

Performanța acestuia este strâns legată de **fine-tuning-ul parametrilor fizici**: variabila **force_mag** determină autoritatea controlului, în timp ce **tau**(prin Integrarea Euler) asigură stabilitatea matematică a simulării.

Mai mult, **designul funcției de recompensă (reward shaping)** s-a dovedit a fi factorul critic; prin ajustarea recompenselor bazate pe starea unghiulară și poziția căruciorului, se poate ghida agentul de la o simplă supraviețuire la un control precis și centrat, crescând semnificativ numărul maxim de pași rezistați.

7. Bibliografie

7.1: <https://en.wikipedia.org/wiki/Q-learning>

7.2: <https://medium.com/swlh/using-q-learning-for-openai-cartpole-v1-4a216ef237df>

7.3: <https://numpy.org/doc/>

7.4: <https://www.pygame.org/docs/>

7.5: <https://docs.python.org/3/library/math.html>

8. O listă cu ce a lucrat fiecare membru al echipei

Duda Andrei-Ionuț:

- Scheletul proiectului, Găsirea parametrilor optimi, Documentatie(Listarea Părților Semnificative, Rezultate Obținute, Concluzii).

Popovici Andrei:

- Contribuții la implementarea algoritmului Q-Learning, testare și ajustare hyperparametrii

Jireadă Teodor:

- Contribuții importante la Interfață, contribuții la implemmentare algoritmului Q-Learning și Q-Table